

Decisiones de Diseño: Árbol de Sintaxis Abstracta (AST)

Agustin Alieni, Francisco Natale y Julian Varea

1. Introducción

En este documento se detallan las decisiones de diseño y refactorizaciones aplicadas sobre la estructura y generación del Árbol de Sintaxis Abstracta (AST), con el objetivo de simplificar y homogeneizar el procesamiento en etapas posteriores.

2. Decisiones de diseño

2.1. Refactorización de la estructura del Nodo AST

Anteriormente, las estructuras que requerían una cantidad variable de elementos utilizaban listas, mientras que los nodos con aridad fija (como operadores binarios con `left` y `right`, o condicionales) tenían atributos específicos.

Para unificar esto, implementamos un sistema genérico basado en parámetros:

- Para nodos de aridad fija, utilizamos índices de parámetros fijos (por ejemplo, en un operador binario, el operando izquierdo es `param[0]` y el derecho es `param[1]`).
- Para nodos de aridad dinámica (listas de sentencias, etc.), guardamos el parámetro en una estructura temporal y luego instanciamos un "nodo lista" en la estructura del nodo. Este nodo lista reserva el lugar y permite ir guardando los sub-nodos.

Esto nos brinda una única interfaz para iterar y limpiar los hijos de los nodos, sin importar si conocíamos su cantidad de antemano.

2.2. Manejo de Tipos de Datos (Data Types)

Se modificó el comportamiento del analizador sintáctico para que, al resolver la producción correspondiente a un tipo de dato (por ejemplo, `int` o `boolean`), devuelva directamente un enumerado que representa el `DataType` (ej. `TYPE_INT`, `TYPE_BOOLEAN`).

2.3. Aplanamiento (Flatten) de Declaraciones de Variables

Cuando nos encontramos con la declaración de múltiples variables en una sola instrucción (por ejemplo, `int x, y, z;`), capturamos el tipo de dato y la lista de identificadores.

Para procesar esto, introducimos la función `flatten_variables_declarations`. Básicamente, antes teníamos la lista de `children` y se requerían pasos adicionales. Ahora, esta función **aplana** la estructura: toma la lista de identificadores y devuelve directamente una lista de nodos individuales, asignándole a cada nodo (ej. un nodo para `x`, otro para `y`, etc.) el tipo de dato correspondiente.

Esto devuelve una lista de nodos ya conformados de manera independiente, evitando tener que realizar este procesamiento de distribución de tipos en etapas posteriores del compilador.

AST - Parser

